

Oppgave 3

I denne oppgaven genereres firkantpulser og en enkel bitsekvens i GNU Octave. Egne funksjoner brukes for å vise hvordan signalene kan bygges opp fra grunnen av.

Oppsett

Signalpakken lastes inn slik at Octaves innebygde `square()`-funksjon kan brukes til sammenligning.

```
% Oppgave 3
% Oppgave 3

pkg load signal;
```

Funksjon: mySquareV1

Denne funksjonen lager firkantpulsen ved å beregne signalets fase i radianer og bruke modulo 2π for å finne posisjonen i hver periode.

```
function [t, out] = mySquareV1(t, f0, d_c, bds=[-1,1], pds=0, cent=false);
    #{
        Denne funksjonen genererer firkantpulsen ved å uttrykke signalets fase i
        radianer og bruke modulo  $2\pi$ .

        t: tidsvektor
        f0: fundamental frekvens
        d_c: duty cycle
        bds: bounds aks grenser (hva høy og lav er i verdi) -> bds(1) = lav verdi, bds(2)
            = høy verdi.
        pds: antall perioder man ønsker, hvis pds=0 så er t_ut = t_in, ellers er t_in
            tilpasset til pds.
        cent: få en sentrert positiv puls omkring origo (alså tidskifte med halv
            pulsperiode, t+t1 hvor t1 = (T*d_c/100)/2)

        t: tidsvektor om pds=0 t_ut = t_in eller så er den tilpasset skalert til ms,
        out: resulterte verdier for y-aksen (amplitude vektor).
    #}

    T = 1/f0;

    if pds != 0;
        t_stop = T*pds;
        idx = find(t >= t_stop, 1)-1;
        t = t(1:idx);
    end;

    d_c = d_c/100;
    rads = t*2*pi*f0;

    if cent;
        rads = t*2*pi*f0 + (2*pi*d_c)/2;
    end;

    pulse_end_rad = 2*pi*d_c;
    n_rad = mod(rads, 2*pi);

    out = n_rad;
    out(n_rad >= pulse_end_rad) = bds(1);
    out(n_rad < pulse_end_rad) = bds(2);

    t = t*1e3; # skaler til ms
end;
```

Funksjon: mySquareV2

Her gjøres den samme typen pulsgenerering direkte i tidsdomenet. Periodetiden T og antall samples per periode bestemmer tidsoppløsningen.

```
function [t, out] = mySquareV2(f0, d_c, spp, bds=[-1,1], pds=5, sh=0);  
    #{  
        Denne funksjonen genererer firkantpulsene direkte i tidsdomenet ved å bruke  
        modulo av periodetiden T.  
  
        f0: fundamental frekvens  
        d_c: duty cycle  
        spp: samples per periode  
        bds: bounds aks grenser (hva høy og lav er i verdi) -> bds(1) = lav verdi, bds(2)  
            ) = høy verdi.  
        pds: antall perioder man ønsker  
        shfit: sh i tid gitt i prosent -> t + T*(sh/100)  
  
        t: resulternde tidsvektor skalert til ms,  
        out: resulternde verdier for y-aksen (amplitude vektor).  
    #}  
  
    d_c = d_c/100;  
    T = 1/f0;  
    dt = T/spp;  
    pulse_end_time = T*d_c;  
  
    t = 0:dt:(T*pds-dt);  
    t_mod = mod(t + T*(sh/100), T);  
  
    out = t_mod;  
    out(t_mod < pulse_end_time) = bds(2);  
    out(t_mod >= pulse_end_time) = bds(1);  
  
    t = t*1e3;  
end;
```

a) Generering og sammenligning av firkantpuls

I del a brukes en grunnfrekvens på 10 kHz og 100 samples per periode. Den egenlagde funksjonen sammenlignes med square()-funksjonen i Octave.

```
% a)  
% 100 sample pr periode ga en god nok firekant puls  
f0 = 10e3;  
T = 1/f0;  
fs = f0*100;  
  
t_start = 0;  
dt = 1/fs;  
t_end = T*4-dt;  
  
d_c = 50; # %  
d_c2 = 20; # %  
  
t = (t_start:dt:t_end);  
  
% sammenligning av mySquareV1 og square funksjonen i Octave  
[_, out_m] = mySquareV1(t, f0, d_c);  
[_, out_m2] = mySquareV1(t, f0, d_c2, [0,1], 0, true);  
  
out = square(t*2*pi*f0, d_c);  
t_1 = T*(d_c2/100)/2;  
  
# shiftet for å få en sentrert puls, dette tilsvarer det som er  
# gjort i mySquareV1 med cent = true.  
  
out2 = square((t+t_1)*2*pi*f0, d_c2);
```

```

out2 = (out2 + 1)/2; # skalere til 0 og 1

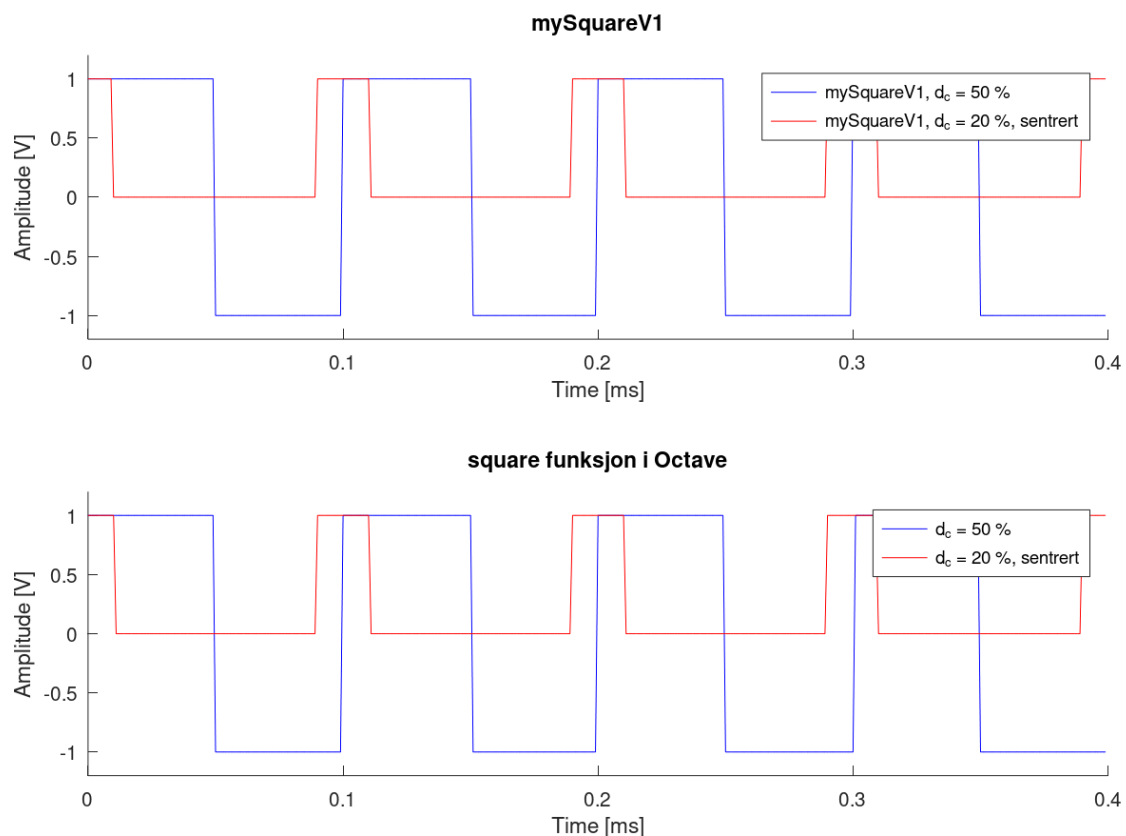
figure(1);

subplot(2,1,1);
hold on;
t = t*1e3; # skaler til ms
plot(t,out_m, "b", "DisplayName", sprintf("mySquareV1, d_c = %d %%", d_c));
plot(t,out_m2, "r", "DisplayName", sprintf("mySquareV1, d_c = %d %, sentrert", d_c2));
ylim([-1.2, 1.2]);
xlabel('Time [ms]');
ylabel('Amplitude [V]');
title(sprintf("mySquareV1"));
legend();

subplot(2,1,2);
hold on;
plot(t, out, "b", "DisplayName", sprintf("d_c = %d %%", d_c));
plot(t, out2, "r", "DisplayName", sprintf("d_c = %d %, sentrert", d_c2));
ylim([-1.2, 1.2]);
xlabel('Time [ms]');
ylabel('Amplitude [V]');
title(sprintf("square funksjon i Octave"));
legend();

```

Resultat a)



Figur 1: Sammenligning mellom den egenlagde firkantpulsen og Octaves square()-funksjon.

Resultatet viser at mySquareV1 gir samme grunnleggende pulsform som Octaves innebygde square()-funksjon. Den røde kurven viser en sentrert puls med 20 % duty cycle.

b) Hjelpfunksjon: addZerosOrOnes

Hjelpenfunksjonen gjør det enklere å bygge bitsekvensen ved å opprette ønsket antall nuller eller ett-tall for hver bit.

```
function y = addZerosOrOnes(x, num);  
    if x == 0;  
        y = zeros(1, num);  
    else  
        y = ones(1, num);  
    end;  
end;
```

Funksjon: bitWaveGen

bitWaveGen setter sammen bitene til ett samlet signal. Parameteren sh brukes til å forskyve sekvensen med en prosentandel av bittiden.

```
function [t, out] = bitWaveGen(bits, T, spp, sh=0)  
    #{  
        bits: vektor med bits (1 og 0) som skal generere en puls  
        T: bit tid (periode) i sekunder  
        spp: samples per periode  
        shfit: sh i tid gitt i prosent -> t + T*(sh/100)  
  
        return verdi  
        t: resulterdne tidsavektor skalert til ms,  
        out: resulterde verdier for y aksen (amplitude vektor).  
    #}  
  
    dt = T/spp;  
    pds = length(bits);  
    sh_num = round(spp*(sh/100));  
  
    t = 0:dt:(T*pds-dt);  
    out = [];  
    first = bits(1);  
  
    for i=1:pds;  
        if i == 1;  
            out = [out, addZerosOrOnes(bits(i), spp-sh_num)];  
        else  
            out = [out, addZerosOrOnes(bits(i), spp)];  
        end;  
    end;  
  
    out = [out, addZerosOrOnes(first, sh_num)];  
  
    t = t*1e3;  
end;
```

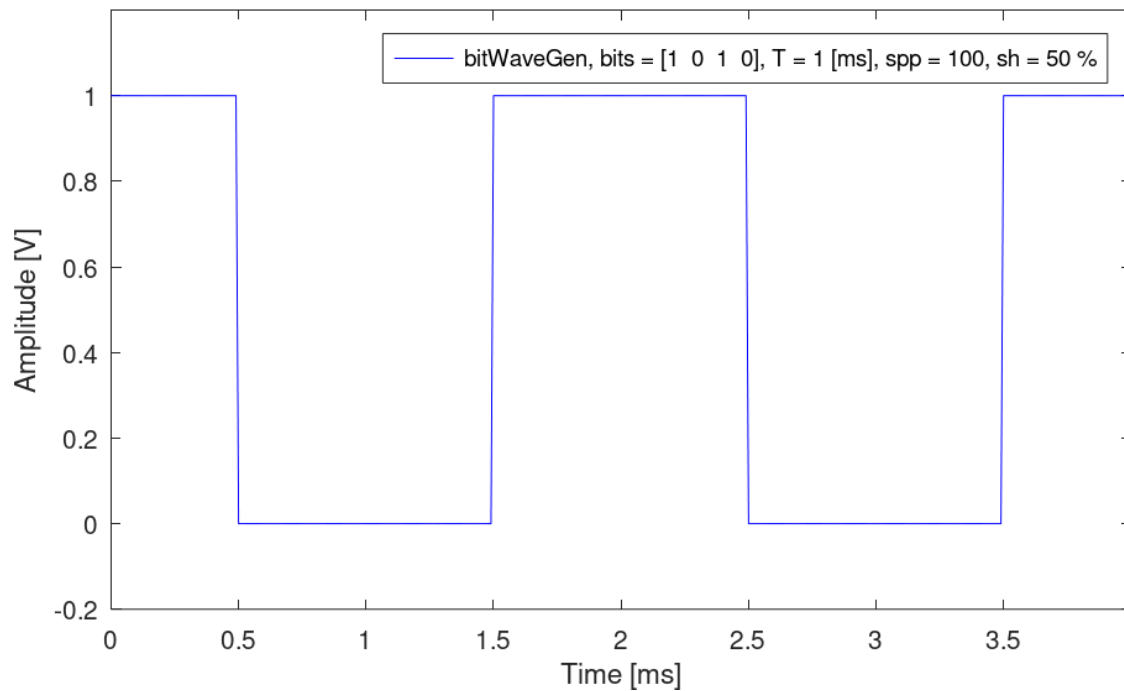
b) Generering av bitsekvens

Her genereres bitsekvensen [1, 0, 1, 0] med en bittid på 1 ms, 100 samples per bit og et shift på 50 %.

```
bits = [1,0,1,0];  
T = 1e-3; # s  
spp = 100; # sample per periode  
sh = 50; # %  
[t, out] = bitWaveGen(bits, T, spp, sh);  
  
figure(3);  
dsp = sprintf("bitWaveGen, bits = [%s], T = %g [ms], spp = %d, sh = %d %%", num2str(bits), T*1e3, spp, sh);  
plot(t, out, "b", "DisplayName", dsp);  
xlabel('Time [ms]');
```

```
ylabel('Amplitude [V]');  
disp(t(end));  
xticks(0:0.5:t(end));  
ylim([-0.2, 1.2]);  
legend();
```

Resultat b)



Figur 2: Den genererte bitsekvensen etter forskyvning.

Figuren viser den ferdige bitsekvensen etter forskyvningen. Hver bit har en varighet på 1 ms, og signalnivået veksler mellom 0 V og 1 V.